

OPERATING SYSTEMS

Comprehensive Study Material

Last Updated: June 30, 2026

TABLE OF CONTENTS

1. Process Management
2. CPU Scheduling
3. Synchronization
4. Memory Management
5. File System
6. Deadlock
7. Inter-Process Communication
8. Threads
9. Shell and System Calls
10. Security Concepts

1. PROCESS MANAGEMENT

Process vs Program: A program is static code on disk. A process is an executing instance with memory, state, and resources. Multiple processes can run the same program.

Process States: New (created), Ready (awaiting CPU), Running (executing), Blocked (waiting for I/O), Terminated (finished). PCB (Process Control Block) maintains process metadata.

Process Control Block: Stores process ID, state, program counter, registers, memory management info, I/O status, and scheduling info. Enables context switching.

Context Switching: OS saves running process state and loads another process. Expensive operation involving cache misses. Goal: balance responsiveness and throughput.

2. CPU SCHEDULING

Scheduling Criteria: CPU utilization, throughput (processes completed per time), turnaround time (total time in system), waiting time (in ready queue), response time (first CPU burst).

FCFS (First-Come-First-Served): Processes execute in arrival order. Simple but causes convoy effect (short jobs wait behind long jobs). No preemption.

SJF (Shortest Job First): Process with shortest CPU burst executes first. Minimizes average waiting time but favors short jobs. Starvation of long processes.

Round Robin: Each process gets time quantum. Preemptive scheduling. Good responsiveness and fairness. Performance depends on quantum size.

Priority Scheduling: Each process has priority level. High-priority processes run first. Can cause starvation. Aging prevents long-waiting processes.

3. SYNCHRONIZATION

Critical Section: Code segment accessing shared resources. Multiple processes executing critical sections simultaneously cause race conditions. Mutual exclusion ensures only one process at a time.

Mutex: Mutual exclusion lock. Binary semaphore (0 or 1). Lock acquired before critical section, released after. Prevents concurrent access.

Semaphores: Integer variable controlling access to shared resources. P (wait) decrements, V (signal) increments. Counting semaphores allow n processes.

Deadlock: Circular wait for resources by multiple processes. No process can proceed. Requires all four conditions: mutual exclusion, hold and wait, no preemption, circular wait.

Deadlock Prevention/Avoidance: Prevent one condition or use safe states. Banker's Algorithm avoids deadlock by careful resource allocation.

4. MEMORY MANAGEMENT

Virtual Memory: Illusion of unlimited memory. Processes see contiguous address space. Actual memory fragmented or on disk. Enables efficient memory utilization.

Paging: Divides physical memory into fixed-size frames and logical address space into pages. Page table maps logical to physical addresses. Eliminates external fragmentation.

Segmentation: Divides logical address space into segments (code, data, stack). Segment table provides base and limit. Matches user view of memory.

Page Replacement: When memory full, choose page to remove. FIFO removes oldest, LRU removes least recently used (best), Optimal removes future-furthest (theoretical).

Thrashing: Excessive page faults when working set exceeds physical memory. Process spends more time paging than computing. Indicates insufficient memory or poor algorithms.

5. FILE SYSTEM

File Allocation: Contiguous allocation stores file in consecutive blocks. Linked allocation uses pointers. Indexed allocation uses index block. FAT and inode use different approaches.

Directory Structure: Single-level (flat), two-level (user directories), tree-structured (hierarchical). Tree structure most common. Path names locate files uniquely.

Disk Scheduling: FCFS serves requests in order. SSTF seeks shortest service time (can starve). SCAN and C-SCAN move disk head systematically. Optimizes throughput and latency.

Inode (Index Node): Data structure storing file metadata and block pointers. Direct pointers for small files, indirect for larger. Unix/Linux file system foundation.

6. DEADLOCK

Necessary Conditions: Mutual exclusion (resource not sharable), hold and wait (holding while requesting), no preemption (cannot forcibly take resource), circular wait (cycle in resource graph).

Deadlock Detection: Build resource allocation graph. Cycle indicates deadlock. Break cycles by preemption or process termination.

Deadlock Prevention: Prevent one condition. Remove mutual exclusion (difficult), break hold and wait (request all upfront), enable preemption, prevent circular wait (order resources).

Deadlock Avoidance: Banker's Algorithm checks if request safe. Only allocates if final state remains safe. Prevents deadlock but requires advance knowledge of resource needs.

7. INTER-PROCESS COMMUNICATION

Pipes: One-way communication between processes. Anonymous pipes (unnamed) for related processes. Named pipes (FIFOs) for unrelated processes on same machine.

Sockets: Endpoints for network communication. TCP sockets (reliable, connection-oriented), UDP sockets (unreliable, connectionless). Enable inter-machine communication.

Message Queues: Asynchronous communication. Processes send messages to queue, receive when ready. Decouples sender and receiver.

Shared Memory: Multiple processes access same memory region. Fastest IPC method. Requires synchronization to prevent race conditions.

8. THREADS

Threads vs Processes: Threads share address space and resources. Multiple threads in one process. Lighter than processes. Context switching cheaper. Sharing simplifies but complicates synchronization.

User vs Kernel Threads: User threads managed by library, kernel unaware. Kernel threads managed by OS. M-to-N model combines benefits. Kernel threads enable true parallelism.

Thread States: New, Runnable, Running, Blocked, Terminated. Transitions managed by scheduler. Yield allows other threads to run.

Thread Synchronization: Ensure safe concurrent access. Mutex, semaphores, monitors. Avoid race conditions, deadlock, starvation.

9. SHELL AND SYSTEM CALLS

Shell: Command-line interpreter. Reads commands, executes programs. Provides job control, I/O redirection, piping. Bash common on Linux.

System Calls: Interface between user programs and OS kernel. Examples: fork (create process), exec (execute), read, write. Bridge application to kernel services.

User vs Kernel Mode: User mode restricted, cannot access hardware. Kernel mode full access. System calls trigger mode switch. Protects OS from user programs.

10. SECURITY CONCEPTS

User Permissions: Read (4), write (2), execute (1). Owner, group, others. Linux chmod command modifies permissions. Principle of least privilege limits access.

Access Control: Discretionary (owner grants), mandatory (system enforces). ACLs (access control lists) specify allowed operations. Role-based access control for organizations.

Authentication: Passwords, multi-factor authentication, biometrics. Verify user identity. Critical for system security and accountability.

Encryption: Protects data confidentiality. File encryption (at-rest), communication encryption (in-transit). Prevents unauthorized access even if storage compromised.